MPOO 2027-1 · GRUPO 6 · PRÁCTICA 11 · MIÉRCOLES 11 DE NOVIEMBRE

La plataforma de streaming

Polimorfismo, segunda parte. Una app como las que usas para oír música y ver series: todo se reproduce, pero no todo se descarga ni todo se califica. La clase abstracta dice qué **es** cada cosa; las interfaces, qué **sabe hacer**.

Descargar los archivos (.zip)

Qué vas a hacer

En pareja (o solo). **Hoy empiezas un proyecto nuevo**: no necesitas el código de la P10. Vas a escribir **9 archivos**: una clase abstracta, dos interfaces, cinco clases y el `Main`, que ya viene en el .zip.

1. **Antes de empezar**
   10 min · la idea de hoy, la tabla de capacidades y el diagrama
2. **Parte A, ejemplo guiado**
   25 min · `Contenido`, la interfaz `Descargable` y la `Cancion`, con el código completo
3. **Parte B, medio guiada**
   20 min · la interfaz `Calificable` y el `Video` vienen completos; la `Pelicula` `final` la escribes tú
4. **Parte C, por tu cuenta**
   20 min · el `Podcast`, con dos interfaces, y la `App`, que no es de la familia
5. **Parte D, tu código en el `Main`**
   15 min · un `try-catch` y un `instanceof`
6. **Parte E, los candados**
   10 min · tres errores a propósito

Cada parte termina en un **punto de control**: corres el `Main` y comparas con la salida de la guía. Lo que no termines en clase lo terminas en casa. La entrega es el **viernes 13 de noviembre, 23:59**, en `p11/`.

Las pistas

Donde escribes tú, la guía trae pistas cerradas, de la más suave a la más directa. **Intenta primero; abre una pista solo si te atoras**, y una a la vez.

▼ **Así se ve una pista**

Aquí adentro va la ayuda. La última pista de cada paso trae el esqueleto con huecos `___` para llenar.

De dónde venimos

YA LO VISTE	DÓNDE	HOY APARECE EN
Clase abstracta, método abstracto, `@Override`	P9 y P10 (`Herramienta`)	`Contenido`
Una variable del tipo de la madre que guarda objetos de las hijas; el `for` que no pregunta	P10 (`Herramienta[]`)	la maratón, Parte C
`instanceof` y cast	P10 (`Moldes`)	Parte D2
`interface`, `implements`, varias interfaces	lunes 9 (`Pato implements Volador, Nadador`)	`Descargable` y `Calificable`
`throw` y `try-catch`	lunes 9 (el menú que no truena)	`calificar` y la Parte D1
`final` en atributos	P9 y P10	**una clase `final`**: el último eslabón

La idea de hoy

Una **clase abstracta** dice qué **es** un objeto; una **interfaz** dice qué **sabe hacer**. Una canción **es** un contenido y **sabe** descargarse. Una app **no es** un contenido, pero **también sabe** descargarse: por eso una canción y una app pueden ir en la misma lista de descargas.

Cada clase de hoy se decide con tres preguntas. Esta tabla es el mapa de la práctica:

CLASE	¿SE REPRODUCE?	¿SE DESCARGA?	¿SE CALIFICA?	POR ESO SU PRIMERA LÍNEA ES

`Cancion`	✔	✔	✘	`public class Cancion extends Contenido implements Descargable`
`Video`	✔	✘	✔	`public class Video extends Contenido implements Calificable`
`Pelicula`	✔	✔	✔ (lo hereda de `Video`)	*la escribes tú en la Parte B*
`Podcast`	✔	✔	✔	*la escribes tú en la Parte C*
`App`	✘	✔	✔	*la escribes tú en la Parte C*

La regla para leerla: ✔ en «se reproduce» se vuelve `extends Contenido`; ✔ en «se descarga», `implements Descargable`; ✔ en «se califica», `implements Calificable`. Si una clase tiene dos interfaces, van separadas por coma.

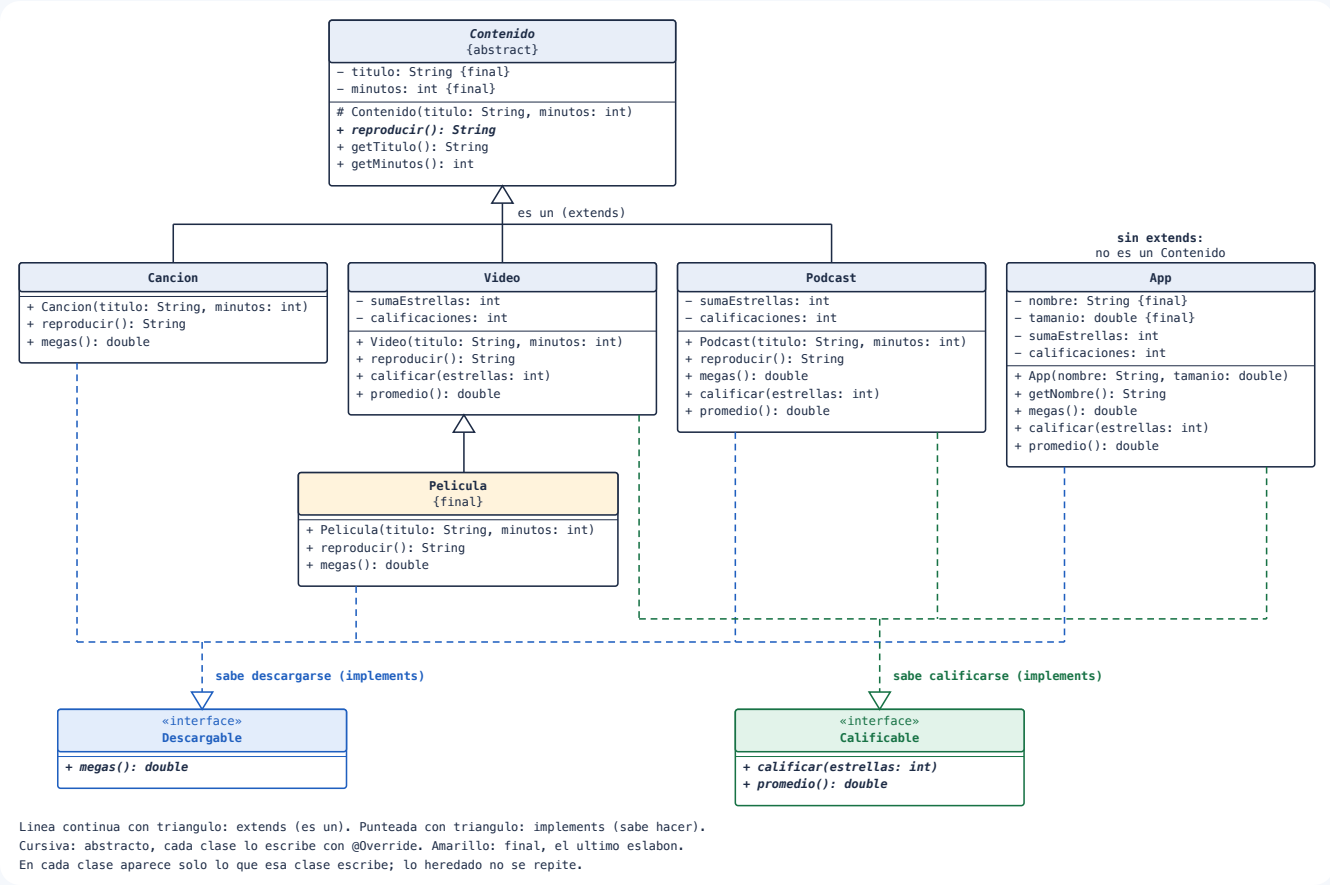
Clase abstracta contra interfaz

	CLASE ABSTRACTA: `CONTENIDO`	INTERFAZ: `DESCARGABLE`
Responde	¿qué **es**?	¿qué **sabe hacer**?
Se declara con	`public abstract class Contenido`	`public interface Descargable`
Una clase la usa con	`extends`: **solo una**	`implements`: **las que quieras**, separadas por coma
¿Guarda datos?	sí: `titulo` y `minutos`	no: solo constantes (`public static final`)
¿Tiene constructor?	sí, y las hijas lo llaman con `super(...)`	no
Sus métodos	abstractos y normales	abstractos; desde Java 8 también `default`, con cuerpo (lo viste el lunes)
¿Se puede hacer `new`?	no	no

> **Ojo con el manual.** La guía 11 del MADO-21 dice que una interfaz es «una clase abstracta pura» y que todos sus métodos son abstractos. Eso era cierto hasta Java 7. Desde Java 8 una interfaz puede tener métodos `default`, con cuerpo, como viste el lunes. La diferencia que sí importa hoy es la de la tabla: **la interfaz no guarda datos ni tiene constructor**.

El diagrama

Así queda la plataforma. Lo de arriba es lo más general y lo de abajo lo más específico: `Contenido`, luego `Video` y al final `Pelicula`, que es `final`. Las líneas **azules** van a `Descargable` y las **verdes** a `Calificable`.



Prepara tu proyecto

1. Crea una carpeta nueva `p11` y adentro `src/streaming/`. En IntelliJ: proyecto nuevo, clic derecho en `src`, **New → Package**, y lo llamas `streaming`.
2. Del .zip, copia `Main.java` dentro de `src/streaming/`.
3. Para crear cada archivo: clic derecho en `streaming`, **New → Java Class**. Para una interfaz, en esa misma ventana eliges **Interface** en vez de **Class**.

```
p11/
  src/streaming/   # Main.java del .zip y tus 8 archivos
```

Para compilar y correr, desde la carpeta `p11` (o con el botón ▶ de IntelliJ sobre el `Main`):

```
javac -d out src/streaming/*.java
# compila todos los archivos de streaming y deja los .class en out
java -cp out streaming.Main
# corre el Main buscando las clases en out
```

En Windows usa `\` en vez de `/` en las rutas.

El `Main` viene con las partes B, C y D apagadas

Cada parte del `Main` está dentro de un comentario `/* ... */`, para que compile aunque todavía no existan sus clases. Cuando termines una parte la «enciendes»: borras la línea que empieza con `/* ===== PARTE` y la línea que termina con `*/`. Las dos dicen «borra esta linea».

Parte A · Ejemplo guiado: el contenido, la primera interfaz y la canción

Esta parte trae el código completo. Escríbelo en tu proyecto (no lo pegues: al teclearlo lo entiendes) y lee lo que explica cada paso.

A1 · Lo que todos son: `Contenido`

clase abstracta **método abstracto** **constructor protected** **atributos final**

Crea `Contenido.java`:

```java
package streaming;

// Contenido: todo lo que se puede reproducir en la plataforma.
// Es abstracta: no existe "un contenido" a secas; siempre es una cancion, un video o un podcast.
public abstract class Contenido {

    private final String titulo;
    private final int minutos;

    protected Contenido(String titulo, int minutos) {
        this.titulo = titulo;
        this.minutos = minutos;
    }

    // Todo contenido se reproduce, pero cada hija dice como
    public abstract String reproducir();

    public String getTitulo() {
        return titulo;
    }

    public int getMinutos() {
        return minutos;
    }
}
```

- **Es abstracta** porque en la plataforma no existe «un contenido» a secas: siempre es una canción, un video o un podcast. Es lo más general de la jerarquía.
- `reproducir()` es **abstracto**: no tiene cuerpo, termina en `;`. Dice «todo contenido se reproduce», pero cada hija dice cómo.
- Los atributos son `final`: el título y los minutos se asignan **una vez**, en el constructor, y ya no cambian.

A2 · Lo que algunos saben hacer: la interfaz `Descargable`

interface **método abstracto implícito**

Crea `Descargable.java` (en IntelliJ: **New → Java Class → Interface**):

```java
package streaming;

// Descargable: lo que se puede bajar al celular para verlo sin internet.
public interface Descargable {

    // Cuantos megas ocupa al bajarlo
    double megas();
}
```

- Dice **una promesa**: «lo que sea descargable sabe decir cuántos megas ocupa». No dice cómo se calcula.
- `megas()` no lleva `public` ni `abstract`: en una interfaz, **todo método sin cuerpo es público y abstracto** solo por estar ahí.
- No tiene atributos ni constructor: la interfaz no sabe cuántos minutos dura nada. Solo promete.

A3 · Es un contenido y sabe descargarse: la `Cancion`

extends **implements** **super(...)** **@Override**

Crea `Cancion.java`:

```
package streaming;

// Cancion: se reproduce y se descarga. No se califica.
public class Cancion extends Contenido implements Descargable {

    public Cancion(String titulo, int minutos) {
        super(titulo, minutos);
    }

    @Override
    public String reproducir() {
        return "Sonando la cancion " + getTitulo();
    }

    // 1 mega por minuto
    @Override
    public double megas() {
        return getMinutos() * 1.0;
    }
}
```

- **Lee la primera línea en voz alta:** «`Cancion` **es un** `Contenido` y **sabe** descargarse». `extends` va primero y `implements` después.
- Por eso tiene **dos obligaciones**: `reproducir()`, que le exige `Contenido`, y `megas()`, que le exige `Descargable`. Las dos con `@Override`.
- El constructor le pasa el título y los minutos a la madre con `super(...)`, como en la P10.

Si se te olvida `megas()`, la interfaz te lo cobra:

```
SALE
src/streaming/Cancion.java:4: error: Cancion is not abstract and does not override abstract
method megas() in Descargable
public class Cancion extends Contenido implements Descargable {
       ^
1 error
```

Y si lo escribes sin `public`: la interfaz lo prometió público, y la clase no puede cerrarlo.

SALE

```
src/streaming/Cancion.java:17: error: megas() in Cancion cannot implement megas() in
Descargable
    double megas() {
           ^
  attempting to assign weaker access privileges; was public
1 error
```

A4 · Punto de control: una canción, tres variables

upcast **tipo de la variable** **tipo del objeto**

Con `Contenido`, `Descargable` y `Cancion` el `Main` ya compila. Esta es su Parte A:

```
        // ===== PARTE A: una cancion, tres variables =====
        Cancion cancion = new Cancion("Lluvia en CU", 4);
        Contenido comoContenido = cancion;       // toda cancion es un contenido
        Descargable comoDescargable = cancion;   // toda cancion es descargable
        System.out.println(comoContenido.reproducir());
        System.out.println("Ocupa " + comoDescargable.megas() + " MB");
        // comoDescargable.reproducir();          // LINEA A: quitale el // en el paso A4
```

```
javac -d out src/streaming/*.java
# compila tus tres archivos y el Main
java -cp out streaming.Main
# corre el Main
```

DEBE SALIR

```
Sonando la cancion Lluvia en CU
Ocupa 4.0 MB
```

Hay **un solo objeto**, una canción, y **tres variables** que lo apuntan. Cada variable deja llamar cosas distintas:

VARIABLE	SU TIPO	EL OBJETO	QUÉ DEJA LLAMAR
`cancion`	`Cancion`	la canción	`reproducir()`, `megas()`, `getTitulo()`…
`comoContenido`	`Contenido`	la misma canción	lo de `Contenido`: `reproducir()`, `getTitulo()`, `getMinutos()`
`comoDescargable`	`Descargable`	la misma canción	**solo** `megas()`

Ahora pruébalo: quítale el `//` a la **LINEA A** y compila.

```
SALE
src/streaming/Main.java:13: error: cannot find symbol
        comoDescargable.reproducir();           // LINEA A: quitale el // en el paso A4
                       ^
  symbol:   method reproducir()
  location: variable comoDescargable of type Descargable
1 error
```

No compila aunque el objeto **sí** sea una canción que se reproduce: el compilador solo mira **el tipo de la variable**, y a una `Descargable` solo se le puede pedir `megas()`. Es la misma regla de los moldes de la P10. **Vuelve a poner el** `//`.

El patrón que acabas de seguir

1. **Primera línea:** `extends` la madre (lo que **es**) e `implements` las interfaces (lo que **sabe hacer**).
2. **Constructor:** llama a `super(...)`.
3. **Un** `@Override` **por cada obligación:** los métodos abstractos de la madre y los de cada interfaz.

Parte B · El video y la película `final`

B1 · La segunda interfaz: `Calificable`

interface **dos métodos**

Crea `Calificable.java`:

```java
package streaming;

// Calificable: lo que la gente puede calificar con estrellas.
public interface Calificable {

    // Recibe de 1 a 5 estrellas; cualquier otro numero se rechaza
    void calificar(int estrellas);

    // El promedio de las estrellas recibidas (0 si nadie ha calificado)
    double promedio();
}
```

B2 · El `Video` (completo)

implements **throw** **IllegalArgumentException**

Crea `Video.java`. Un video de la plataforma se ve en línea: se reproduce y se califica, pero no se descarga.

```
package streaming;

// Video: se reproduce y se califica. No se descarga: se ve en linea.
public class Video extends Contenido implements Calificable {

    private int sumaEstrellas;
    private int calificaciones;

    public Video(String titulo, int minutos) {
        super(titulo, minutos);
    }

    @Override
    public String reproducir() {
        return "Reproduciendo el video " + getTitulo();
    }

    @Override
    public void calificar(int estrellas) {
        if (estrellas < 1 || estrellas > 5) {
            throw new IllegalArgumentException("calificacion invalida: " + estrellas + "
(debe ir de 1 a 5)");
        }
        sumaEstrellas += estrellas;
        calificaciones++;
    }

    @Override
    public double promedio() {
        if (calificaciones == 0) {
            return 0;
        }
        return (double) sumaEstrellas / calificaciones;
    }
}
```

- **Los dos atributos de las estrellas** los lleva la clase, no la interfaz: `Calificable` solo promete `calificar` y `promedio`; quien los cumple tiene que guardar la suma y cuántas van.
- `calificar` **usa** `throw`, como el lunes: si las estrellas no van de 1 a 5, **no las suma**. Lanza una `IllegalArgumentException` con un mensaje y el método se corta ahí. Quien llamó decide qué hacer (eso lo haces tú en la Parte D).
- `promedio` convierte a `double` antes de dividir, para que 9 / 2 dé 4.5 y no 4. Si nadie ha calificado, devuelve 0 en vez de dividir entre cero.

B3 · Tu turno: la `Pelicula`, el último eslabón

herencia en tres niveles | clase final | implements

Una película **es un** video (se reproduce y se califica), pero **además** se descarga. Y es el último nivel: nadie hereda de una película. Crea `Pelicula.java` con esta tarjeta:

Tarjeta · `Pelicula`

Es un	`Video` (no directamente un `Contenido`)
Además sabe	descargarse
Es el último eslabón	sí: lleva `final`
`reproducir()` devuelve	`"Proyectando la pelicula " + getTitulo()`
`megas()`	25 megas por minuto (calidad HD): `getMinutos() * 25.0`
`calificar` y `promedio`	**no los escribas**: ya los tiene por ser un `Video`

▼ **Pista 1 · ¿De quién hereda?**

Una película es un **video**: `extends Video`, no `extends Contenido`. Así se lleva gratis `calificar`, `promedio` y sus dos atributos. Lo único que agrega es lo que el video no sabe: descargarse.

▼ **Pista 2 · ¿Dónde va el `final`?**

Antes de `class`: `public final class Pelicula ...`. Una clase `final` no puede tener hijas.

▼ **Pista 3 · El esqueleto**

```
package streaming;

public final class Pelicula extends ___ implements ___ {

    public Pelicula(String titulo, int minutos) {
        super(___, ___);
    }

    @Override
    public String reproducir() {
        return ___;
    }

    @Override
    public double megas() {
        return ___;
    }
}
```

El error típico: si pones `extends Contenido`, la película ya no sabe calificarse y el `Main` no compila en `peli.calificar(5)`:

```
SALE
src/streaming/Main.java:19: error: cannot find symbol
        peli.calificar(5);
            ^
  symbol:   method calificar(int)
  location: variable peli of type Pelicula
src/streaming/Main.java:23: error: cannot find symbol
        System.out.println("Pelicula: " + peli.promedio() + " estrellas, ocupa " +
peli.megas() + " MB");
                                           ^
  symbol:   method promedio()
  location: variable peli of type Pelicula
2 errors
```

B4 · Punto de control: enciende la Parte B del `Main`

Borra estas dos líneas del `Main` (la de arriba y la de abajo de la Parte B):

```
/* ===== PARTE B: borra esta linea cuando termines la Parte B =====
...
===== fin de la PARTE B: borra esta linea ===== */
```

Queda así:

```
        Video tutorial = new Video("Tutorial de Git", 12);
        Pelicula peli = new Pelicula("El ultimo commit", 110);
        tutorial.calificar(5);
        tutorial.calificar(4);
        peli.calificar(5);
        System.out.println(tutorial.reproducir());
        System.out.println(peli.reproducir());
        System.out.println("Tutorial: " + tutorial.promedio() + " estrellas");
        System.out.println("Pelicula: " + peli.promedio() + " estrellas, ocupa " +
peli.megas() + " MB");
```

```
javac -d out src/streaming/*.java
# compila con el video y la película
java -cp out streaming.Main
# corre el Main
```

DEBE SALIR

```
Sonando la cancion Lluvia en CU
Ocupa 4.0 MB
Reproduciendo el video Tutorial de Git
Proyectando la pelicula El ultimo commit
Tutorial: 4.5 estrellas
Pelicula: 5.0 estrellas, ocupa 2750.0 MB
```

Fíjate en `peli.calificar(5)`: tú no escribiste `calificar` en `Pelicula`, y funciona. **Lo heredó de `Video`**. Por eso `Pelicula` es `Calificable` sin decirlo: lo es su madre.

Parte C · Por tu cuenta: el podcast y la app

C1 · El `Podcast`: dos interfaces

extends **dos implements**

Tarjeta · `Podcast`

Es un	`Contenido`
Sabe	descargarse **y** calificarse: **dos interfaces**
`reproducir()` devuelve	`"Escuchando el podcast " + getTitulo()`
`megas()`	Medio mega por minuto (solo audio): `getMinutos() * 0.5`
`calificar` y `promedio`	iguales a los de `Video`, con sus dos atributos

▼ **Pista 1 · La primera línea**

Es un contenido que sabe **dos** cosas: `extends Contenido implements Descargable, Calificable`. Las interfaces van separadas por coma.

▼ **Pista 2 · ¿Cuántos métodos?**

Cinco: el constructor; `reproducir()`, que le exige `Contenido`; `megas()`, que le exige `Descargable`; y `calificar` y `promedio`, que le exige `Calificable`. Los dos últimos, con sus dos atributos, son **iguales a los de `Video`**: cópialos.

▼ **Pista 3 · El esqueleto**

```
package streaming;

public class Podcast extends ___ implements ___, ___ {

    private int sumaEstrellas;
    private int calificaciones;

    public Podcast(String titulo, int minutos) {
        ___
    }

    @Override
    public String reproducir() { ___ }

    @Override
    public double megas() { ___ }

    // calificar y promedio: iguales a los de Video
}
```

C2 · La `App`: fuera de la familia

`implements sin extends` `this`

Tarjeta · `App`

Es un	**nada de la familia**: una app no se reproduce, así que **no** es un `Contenido`
Sabe	descargarse y calificarse
Atributos	`nombre` (`String`, `final`), `tamanio` (`double`, `final`), y los dos de las estrellas
Constructor	`App(String nombre, double tamanio)`: guarda los dos con `this.`
Métodos	`getNombre()`; `megas()` devuelve `tamanio`; `calificar` y `promedio` iguales a los de `Video`

<details>
<summary>▼ Pista 1 · Sin <code>extends</code></summary>

Una app no se reproduce, así que no es un contenido: su primera línea es `public class App implements Descargable, Calificable`. Sin `extends`.
</details>

<details>
<summary>▼ Pista 2 · Sin <code>super</code></summary>

Como no hereda de `Contenido`, no hay `super(titulo, minutos)` ni `getTitulo()`. El constructor guarda sus propios atributos: `this.nombre = nombre;` y `this.tamanio = tamanio;`. Y `megas()` solo devuelve `tamanio`.
</details>

▼ **Pista 3 · El esqueleto**

```
package streaming;

public class App implements ___, ___ {

    private final String nombre;
    private final double tamanio;
    private int sumaEstrellas;
    private int calificaciones;

    public App(String nombre, double tamanio) {
        ___
        ___
    }

    public String getNombre() { ___ }

    @Override
    public double megas() { ___ }

    // calificar y promedio: iguales a los de Video
}
```

¿Por qué copiar `calificar` tres veces?

Porque la interfaz solo **promete**: no puede guardar la suma de estrellas. `Video`, `Podcast` y `App` la cumplen cada una por su lado. `Pelicula` no la copia porque la hereda de `Video`.

C3 · Punto de control: la maratón y las descargas

referencia del tipo más general **referencia de tipo interfaz** **enlace dinámico**

Enciende la Parte C del `Main` igual que la B. Trae dos arreglos:

```
        Podcast podcast = new Podcast("Cafe con Java", 40);
        App mapa = new App("Mapa de CU", 80);
        mapa.calificar(4);
        mapa.calificar(2);
        System.out.println(mapa.getNombre() + ": " + mapa.promedio() + " estrellas");

        // La maraton: un arreglo del tipo mas general de la familia, Contenido
        System.out.println("-- Maraton --");
        Contenido[] maraton = { cancion, tutorial, peli, podcast };
        int minutos = 0;
        for (Contenido c : maraton) {
            System.out.println(c.reproducir());
            minutos += c.getMinutos();
        }
        System.out.println("Total: " + minutos + " minutos");

        // Las descargas: un arreglo de tipo interfaz; mezcla clases sin padre comun
        System.out.println("-- Descargas --");
        Descargable[] descargas = { cancion, peli, podcast, mapa };
        double megas = 0;
        for (Descargable d : descargas) {
            megas += d.megas();
        }
        System.out.println("Total: " + megas + " MB");
        if (megas <= 2048) {
            System.out.println("Cabe en tus 2 GB de datos");
        } else {
            System.out.println("No cabe en tus 2 GB de datos: sobran " + (megas - 2048) + "
MB");
        }
```

- `Contenido[] maraton` es un arreglo del **tipo más general de la familia**, con objetos de cuatro clases. El `for` llama a `c.reproducir()` sin preguntar qué es cada uno, y cada objeto responde a su manera: es el enlace dinámico de la P10.
- `Descargable[] descargas` es un arreglo **de tipo interfaz**. Mezcla una canción, una película, un podcast y **la app, que no es de la familia**. Lo único que tienen en común es que saben decir sus megas.

```
javac -d out src/streaming/*.java
# compila con el podcast y la app
java -cp out streaming.Main
# corre el Main
```

DEBE SALIR

```
Sonando la cancion Lluvia en CU
Ocupa 4.0 MB
Reproduciendo el video Tutorial de Git
Proyectando la pelicula El ultimo commit
Tutorial: 4.5 estrellas
Pelicula: 5.0 estrellas, ocupa 2750.0 MB
Mapa de CU: 3.0 estrellas
-- Maraton --
Sonando la cancion Lluvia en CU
Reproduciendo el video Tutorial de Git
Proyectando la pelicula El ultimo commit
Escuchando el podcast Cafe con Java
Total: 166 minutos
-- Descargas --
Total: 2854.0 MB
No cabe en tus 2 GB de datos: sobran 806.0 MB
```

Las cuentas: la maratón suma 4 + 12 + 110 + 40 = 166 minutos. Las descargas, 4 + 2750 + 20 + 80 = 2854 MB, y no caben en 2048.

Pruébalo: agrega `mapa` al arreglo `maraton` y compila. No cabe: la app no es un contenido.

SALE

```
src/streaming/Main.java:33: error: incompatible types: App cannot be converted to Contenido
        Contenido[] maraton = { cancion, tutorial, peli, podcast, mapa };
                                                                  ^
1 error
```

Quítala. **En la lista de descargas sí cabe y en la maratón no**: eso es lo que una interfaz te deja hacer y la herencia no.

Parte D · Tu código en el `Main`

Enciende la Parte D. Tiene dos huecos que dicen `TU CODIGO`:

```
        // D1: las calificaciones del grupo para el podcast. Dos no valen.
        System.out.println("-- Calificaciones del podcast --");
        int[] estrellas = { 5, 6, 4, 0, 3 };
        for (int e : estrellas) {
            // TU CODIGO (D1)
        }
        System.out.println("Podcast: " + podcast.promedio() + " estrellas");

        // D2: de la maraton, que se califica y que no
        System.out.println("-- Que se califica --");
        for (Contenido c : maraton) {
            // TU CODIGO (D2)
        }
```

D1 · Las calificaciones del grupo: `try-catch`

`try-catch` `getMessage()`

El grupo calificó el podcast con `5, 6, 4, 0, 3`. El 6 y el 0 no valen. Primero haz la prueba ingenua: en el hueco D1 escribe solo `podcast.calificar(e);`, compila y corre. El programa **truena en el 6** y ya no sigue:

SALE

```
Exception in thread "main" java.lang.IllegalArgumentException: calificacion invalida: 6 (debe
ir de 1 a 5)
at streaming.Podcast.calificar(Podcast.java:27)
at streaming.Main.main(Main.java)
```

Es lo mismo que el menú del lunes: una entrada mala tira todo el programa. **Tu tarea:** que el 6 y el 0 se rechacen con un aviso y el programa siga con las demás. El aviso es `"Rechazada: "` más el mensaje de la excepción.

▼ **Pista 1 · La forma**

`try` envuelve lo que puede fallar; `catch` dice qué hacer si falla. Si no falla, el `catch` no corre.

▼ **Pista 2 · El tipo y el mensaje**

La traza dice qué excepción es: `IllegalArgumentException`. Si la atrapas como `ex`, su mensaje sale con `ex.getMessage()`.

▼ **Pista 3 · El esqueleto**

```
for (int e : estrellas) {
    try {
        ___
    } catch (___ ex) {
        System.out.println("Rechazada: " + ___);
    }
}
```

D2 · ¿Qué se califica?: `instanceof`

`instanceof` `downcast`

En la maratón, unos contenidos se califican y otros no. Por cada `c` de la maratón: si es `Calificable`, imprime su título y su promedio; si no, su título y `no se califica`. Mira la salida esperada abajo para el formato exacto.

▼ **Pista 1 · La pregunta**

`c instanceof Calificable` pregunta por el **objeto**, no por la variable: da `true` para el video, la película y el podcast, y `false` para la canción.

▼ **Pista 2 · Llegar a `promedio()`**

La variable `c` es `Contenido`, y `Contenido` no tiene `promedio()`. Necesitas una variable `Calificable`: un downcast, como en la P10: `Calificable k = (Calificable) c;`. Java 21 deja hacer la pregunta y el molde en una sola línea: `if (c instanceof Calificable k)`, y adentro del `if` ya existe `k`. Cualquiera de las dos vale.

▼ **Pista 3 · El esqueleto**

```
for (Contenido c : maraton) {
    if (c instanceof ___ k) {
        System.out.println(c.getTitulo() + ": " + ___ + " estrellas");
    } else {
        System.out.println(___ + ": no se califica");
    }
}
```

> **¿No que `instanceof` era lo malo?**
>
> En la P10 la cadena de `instanceof` preguntaba **«¿qué clase eres?»** para decidir por el objeto cómo golpear: eso lo resuelve el polimorfismo. Aquí preguntas **«¿sabes calificarte?»**, una capacidad que no todos tienen. La alternativa sería poner `promedio()` en `Contenido`, y entonces una canción tendría que inventarse estrellas.

D3 · Punto de control final

Compila y corre. Debe salir todo lo anterior y, al final:

```
DEBE SALIR
-- Calificaciones del podcast --
Rechazada: calificacion invalida: 6 (debe ir de 1 a 5)
Rechazada: calificacion invalida: 0 (debe ir de 1 a 5)
Podcast: 4.0 estrellas
-- Que se califica --
Lluvia en CU: no se califica
Tutorial de Git: 4.5 estrellas
El ultimo commit: 5.0 estrellas
Cafe con Java: 4.0 estrellas
```

`El ultimo commit` sale con 5.0 aunque `Pelicula` no dice `implements Calificable`: lo heredó de `Video`.

Parte E · Los candados

Tres errores a propósito. En cada uno: lo provocas, lees el mensaje y lo deshaces (**Ctrl+Z**). Toma captura de cada error para la bitácora.

E1 · Nadie hereda de una película

`final`

Crea `PeliculaDeTerror.java` así y compila:

```
package streaming;

public class PeliculaDeTerror extends Pelicula {

    public PeliculaDeTerror(String titulo, int minutos) {
        super(titulo, minutos);
    }
}
```

```
SALE
src/streaming/PeliculaDeTerror.java:3: error: cannot inherit from final Pelicula
public class PeliculaDeTerror extends Pelicula {
                                      ^
1 error
```

`final` cierra la jerarquía: `Pelicula` es el último eslabón. ¿Y si quieres películas de terror? **El género es un dato, no una clase**: sería un atributo `genero` en `Pelicula`. **Borra el archivo.**

E2 · Una interfaz no se instancia

En el `Main`, antes de la canción, agrega `Descargable algo = new Descargable();` y compila:

```
SALE
src/streaming/Main.java:8: error: Descargable is abstract; cannot be instantiated
        Descargable algo = new Descargable();
                           ^
1 error
```

Una interfaz es una **promesa**, no una cosa: no hay «un descargable» a secas, como no hay «un contenido» a secas. **Borra la línea.**

E3 · La interfaz cobra

En `Podcast`, borra el método `megas()` completo y compila:

```
SALE
src/streaming/Podcast.java:4: error: Podcast is not abstract and does not override abstract
method megas() in Descargable
public class Podcast extends Contenido implements Descargable, Calificable {
       ^
1 error
```

El `Podcast` dijo `implements Descargable`: prometió `megas()` y no lo cumplió. **Deshazlo con Ctrl+Z.**

Si terminas: retos

- **El audiolibro:** escribe `Audiolibro extends Podcast` (se reproduce como `"Leyendo el audiolibro "`) y agrégalo a la maratón. Luego intenta `Audiolibro extends Pelicula`: ¿por qué no se puede?
- **Un `default` útil:** agrega a `Descargable` el método `default boolean cabeEn(double libres)`, que devuelva `megas() <= libres`. Pregúntale a cada descarga si cabe en 500 MB. ¿Cuántas clases tuviste que cambiar?
- **El choque de dos `default`:** agrega a **las dos** interfaces un `default String resumen()`, una que devuelva `"se descarga"` y otra `"se califica"`. Compila:

```
SALE
src/streaming/App.java:5: error: types Descargable and Calificable are incompatible;
public class App implements Descargable, Calificable {
       ^
  class App inherits unrelated defaults for resumen() from types Descargable and Calificable
src/streaming/Pelicula.java:5: error: types Descargable and Calificable are incompatible;
public final class Pelicula extends Video implements Descargable {
            ^
  class Pelicula inherits unrelated defaults for resumen() from types Descargable and
Calificable
src/streaming/Podcast.java:4: error: types Descargable and Calificable are incompatible;
public class Podcast extends Contenido implements Descargable, Calificable {
       ^
  class Podcast inherits unrelated defaults for resumen() from types Descargable and
Calificable
3 errors
```

Las clases que tienen las dos interfaces no saben cuál usar. Arréglalo en `Podcast` y en `App` con `@Override public String resumen()` que devuelva `Descargable.super.resumen() + " y " + Calificable.super.resumen()`. En `Pelicula` eso no compila, porque `Calificable` no es interfaz suya sino de su madre: ahí usa `super.resumen()` para la parte de `Video`.

Si algo truena

No compila

- **`... is not abstract and does not override abstract method ...`**: a esa clase le falta un método que le exige su madre o una de sus interfaces. El mensaje dice cuál y de quién.
- **`attempting to assign weaker access privileges; was public`**: escribiste un método de la interfaz sin `public`.
- **`cannot find symbol` en `calificar` o `promedio`**: esa clase no sabe calificarse. ¿Tu `Pelicula` dice `extends Video`? ¿Tu `Podcast` dice `implements ..., Calificable`?
- **`cannot find symbol` en `tutorial`, `peli`, `podcast` o `mapa`**: encendiste una parte del `Main` antes que la anterior. Se encienden en orden: B, C, D.
- **`... is abstract; cannot be instantiated`**: hay un `new` de `Contenido`, `Descargable` o `Calificable`.
- **`illegal start of expression` justo al final de una parte del `Main`**: borraste la línea de arriba de esa parte pero no la de abajo. Borra las dos.

Compila, pero algo sale distinto

- **Un promedio sale 4.0 en vez de 4.5**: dividiste dos `int`. Convierte a `double` antes de dividir.
- **El podcast sale con 3.6 estrellas**: el 6 y el 0 se sumaron. Falta la condición en `calificar` (paso B2).
- **El programa se detiene en «Exception in thread main»**: la llamada a `calificar` de la Parte D1 no está dentro del `try`.
- **Encendiste una parte y no sale nada nuevo**: borraste la línea de abajo pero no la de arriba, y el comentario se sigue hasta la parte siguiente. Borra las dos.
- **Las descargas no dan 2854.0**: revisa las reglas de megas de las tarjetas: 1, 25 y 0.5 por minuto, y la app su tamaño.

La entrega

QUÉ	DETALLE
Cuándo	**Viernes 13 de noviembre, 23:59.** Sin IA, como siempre.
Con quién	En pareja (o solo). **Cada integrante sube todo en su propia carpeta.**
Dónde	Tu rama `entregas_apellido_nombre`, carpeta `entregas/apellido_nombre/p11/`. Solo push.
Qué	Tu carpeta `src` y una **bitácora en PDF**

La bitácora lleva: los integrantes (nombre y número de cuenta); captura de la salida completa del `Main`; capturas del error de la LINEA A y de los tres candados (E1, E2 y E3); y estas tres respuestas con tus palabras:

1. **Una canción, tres variables:** ¿por qué la LINEA A no compila si el objeto es una canción que sí se reproduce? ¿Quién lo detectó, el compilador o Java al correr?
2. **La película:** `Pelicula` no dice `implements Calificable`, pero en la Parte D2 sale con su promedio. ¿Por qué? ¿Por qué no compila `PeliculaDeTerror`, y cómo guardarías el género de una película sin heredar?
3. **La app:** ¿por qué `mapa` cabe en `Descargable[]` pero no en `Contenido[]`? Con tus palabras: ¿qué te deja hacer una interfaz que no te deja la herencia?

Cómo se califica, sobre 10:

QUÉ REVISO	PUNTOS
Fiel al diagrama: `Contenido` abstracta; las dos interfaces; cada clase con el `extends` y los `implements` de la tabla de capacidades; `Pelicula` `final` y heredando de `Video`; `App` sin `extends`; `@Override` en cada obligación	3.5

Corre: el `Main` da la salida completa de la guía	3
Parte D: el `try-catch` rechaza el 6 y el 0 y el programa sigue; el `instanceof` separa lo que se califica	1.5
Bitácora: las capturas y las tres respuestas	2

Cada comando de abajo, con lo que hace:

```
git checkout entregas_apellido_nombre
# te pasa a tu rama de entregas
git pull origin main
# trae lo nuevo del repositorio del grupo
mkdir -p entregas/apellido_nombre/p11
# crea tu carpeta de esta práctica; copia ahí src y la bitácora en PDF
git add entregas/apellido_nombre/p11
# marca lo que vas a entregar
git commit -m "P11: polimorfismo, segunda parte"
# lo guarda con un mensaje
git push
# lo sube: esto es la entrega
```

Si los `.class` de `out` se cuelan, no pasa nada. El código tiene que ser suyo; si algo no sale, entréguenlo igual y cuenten en la bitácora dónde se atoraron.

Modelos de Programación Orientada a Objetos 2027-1 · Grupo 6 · Facultad de Ingeniería, UNAM
Prof. Oscar Manuel Ruiz Hurtado · Práctica 11 · Polimorfismo, segunda parte (MADO-21, guía 11: jerarquía de clases desde lo muy abstracto hasta lo muy específico, con `Contenido`, `Video` y la clase `final` `Pelicula`; referencia de la clase base más general; instancias de las clases derivadas; interfaces) · noviembre de 2026